Servicio Nacional De Aprendizaje

Curso De Formación Complementaria
Desarrollo De Aplicaciones Con Interfaz Grafica, Manejo De Eventos, Clases Y Objetos: Java

Actividad 1:
Clases y Objetos

Instructor
Alexis Vergara Arrieta

Aprendiz
Cristian Andrés Basto Largo

Ficha: 3433095

2026

Modelación

En programación orientada a objetos se utilizan diagramas UML, los cuales permiten representar gráficamente las clases, sus atributos, métodos y relaciones.

Las clases serían como los elementos principales del sistema, el molde para crear objetos.

Las clases para la actividad serán:

- Persona
- Cliente
- Empleado
- Cuenta
- CuentaAhorro
- CuentaInversion

Cada clase tendrá sus atributos y métodos.

Las variables de instancia pertenecen a cada objeto creado a partir de una clase, por ejemplo el nombre de un cliente o el saldo de una cuenta. Las variables static pertenecen a la clase en general y son compartidas por todos los objetos, por ejemplo un contador del número total de cuentas creadas.

Y los comportamientos de cada objeto se representarán con los métodos como depositar dinero, retirar dinero y mostrar el saldo.

Como tal las clases principales serán Persona y Cuenta y de estas clases saldrán las clases Cliente y Empleado, y CuentaAhorro y CuentaInversion, esas clases van a heredar los atributos y métodos de sus clases padre (Persona y Cuenta). La clase BancoApp es donde estará el método main y esa gestión del sistema.

Para la clase Persona tenemos:

actividad1 > personas > Persona.java > ...

```java
package personas;

public abstract class Persona {

    protected int numero;
    protected String nombre;

    public Persona(int numero, String nombre) {
        this.numero = numero;
        this.nombre = nombre;
    }

    public void mostrarDatos() {
        System.out.println("Numero: " + numero);
        System.out.println("Nombre: " + nombre);
    }
}
```

Y para las clases que van a heredar de esta que son Empleado y Cliente:

actividad1 > personas > Cliente.java > Cliente

```java
package personas;

public class Cliente extends Persona {

    public Cliente(int numero, String nombre) {
        super(numero, nombre);
    }
}
```

actividad1 > personas > Empleado.java > Empleado > puesto

```java
package personas;

public class Empleado extends Persona {

    private String puesto;
    private int aniosTrabajando;

    public Empleado(int numero, String nombre, String puesto, int aniosTrabajando) {
        super(numero, nombre);
        this.puesto = puesto;
        this.aniosTrabajando = aniosTrabajando;
    }

    public int calcularVacaciones() {

        int dias = 5 + (aniosTrabajando * 2);

        if (dias > 20) {
            dias = 20;
        }

        return dias;
    }
}
```

Ahora para la parte de las cuentas tendríamos la clase padre Cuenta:

actividad1 > cuentas > Cuenta.java > Cuenta

```java
package cuentas;

import personas.Cliente;

public abstract class Cuenta {

    protected int numeroCuenta;
    protected double saldo;
    protected Cliente cliente;

    public static int totalCuentas = 0;

    public Cuenta(int numeroCuenta, Cliente cliente, double saldoInicial) {
        this.numeroCuenta = numeroCuenta;
        this.cliente = cliente;
        this.saldo = saldoInicial;
        totalCuentas++;
    }

    public void depositar(double monto) {
        saldo += monto;
    }

    public abstract void retirar(double monto);

    public void mostrarSaldo() {
        System.out.println("Saldo actual: " + saldo);
    }
}
```

Y sus clases hijas, cuentaAhorro y CuentaInversion

actividad1 > cuentas > CuentaAhorro.java > CuentaAhorro

```java
package cuentas;

import personas.Cliente;

public class CuentaAhorro extends Cuenta {

    private double interesAnual;

    public CuentaAhorro(int numeroCuenta, Cliente cliente, double saldoInicial, double interesAnual) {
        super(numeroCuenta, cliente, saldoInicial);
        this.interesAnual = interesAnual;
    }

    public void retirar(double monto) {

        if ((saldo - monto) >= 500) {
            saldo -= monto;
            System.out.println(x: "Retiro realizado");
        } else {
            System.out.println(x: "No se puede retirar, saldo mínimo 500");
        }
    }

    public void calcularInteres() {
        saldo += saldo * interesAnual / 12;
    }
}
```

actividad1 > cuentas > CuentaInversion.java > CuentaInversion

```java
package cuentas;

import personas.Cliente;

public class CuentaInversion extends Cuenta {

    public CuentaInversion(int numeroCuenta, Cliente cliente, double saldoInicial) {
        super(numeroCuenta, cliente, saldoInicial);
    }

    public void retirar(double monto) {

        if ((saldo - monto) >= 10000) {
            saldo -= monto;
            System.out.println(x: "Retiro realizado");
        } else {
            System.out.println(x: "No se puede retirar, saldo mínimo 10000");
        }
    }
}
```

Y el controller o método main estará en la clase central llamada BancoApp:

```
actividad1 > BancoApp.java > BancoApp
1  import personas.Cliente;
2  import cuentas.CuentaAhorro;
3  import cuentas.CuentaInversion;
4
5  public class BancoApp {
6
       Run | Debug
7      public static void main(String[] args) {
8
9          Cliente cliente1 = new Cliente(numero: 1, nombre: "Juan Perez");
10
11         CuentaAhorro ahorro = new CuentaAhorro(numeroCuenta: 101, cliente1, saldoInicial: 1000, interesAnual: 0.05);
12
13         CuentaInversion inversion = new CuentaInversion(numeroCuenta: 201, cliente1, saldoInicial: 25000);
14
15         ahorro.depositar(monto: 500);
16         ahorro.retirar(monto: 200);
17
18         inversion.retirar(monto: 5000);
19
20         ahorro.mostrarSaldo();
21         inversion.mostrarSaldo();
22
23         System.out.println("Total cuentas creadas: " + cuentas.Cuenta.totalCuentas);
24     }
25 }
```

Son clases abstractas Persona y Cuenta para manejar el polimorfismo, no existe como tal una persona o una cuenta sino que siempre van a ser o un empleado/cliente o una cuenta de ahorro/inversion. El polimorfismo aparece cuando las clases implementan un mismo método de forma diferente.

Al ejecutar el código sale una respuesta basado en los datos quemados puestos:

```
C:\Users\crist\Desktop\Cursos\Desarrollo de Apps con UI, Eventos, POO Java - SENA\projects> cmd
ailsInExceptionMessages -cp C:\Users\crist\AppData\Roaming\Code\User\workspaceStorage\ddff737563
BancoApp "
Retiro realizado
Retiro realizado
Saldo actual: 1300.0
Saldo actual: 20000.0
Total cuentas creadas: 2

C:\Users\crist\Desktop\Cursos\Desarrollo de Apps con UI, Eventos, POO Java - SENA\projects>
```